

Assignment Report #7

Gravitational Potential Calculation for Star Clusters

Student Name: [Adilet Akimshe] (ID: [202194337])

Course: PHYS 421 Parallel Computing

Submitted: November 28, 2025

Software packages used:

CUDA Toolkit 11.x/12.x	GPU kernel development
Python 3.10	Main scripting and automation
CuPy (CUDA 11.x/12.x)	GPU-accelerated NumPy
PyTorch (Latest)	Deep learning framework
NumPy, Matplotlib	Data processing and visualization

AI tools used:

Claude Code	Report writing, data analysis, code review
-------------	--

Abstract

This study compares three GPU-accelerated methods for computing gravitational potential energy in N-body star cluster simulations: Direct Summation, Monte Carlo approximation, and Deep Learning. Each method was implemented in both CUDA C and CuPy (Python) to evaluate low-level versus high-level GPU programming trade-offs. Experiments were conducted on Plummer sphere distributions with $N = 10^3$ to 10^6 particles, varying softening parameters ($\epsilon = 0.001$ to 0.1) and Monte Carlo sample counts ($K = 10^3$ to 10^5). Results show that CUDA Direct Summation achieves optimal performance (1.43s for $N = 10^6$) with highest accuracy (0.12% error), outperforming CuPy by $92\times$. Monte Carlo methods exhibited systematic error dominated by softening parameter choice rather than statistical sampling variance, contradicting the expected $1/\sqrt{K}$ error scaling. The Deep Learning approach failed catastrophically with $> 10^{13}\%$ error due to generalization failure across different particle counts, despite reasonable training performance. CUDA implementations consistently demonstrated superior performance, with the key trade-off being development complexity versus runtime efficiency.

1 Introduction and Problem Statement

1.1 Physical Background

The gravitational potential energy U of an N -body system quantifies the total interaction energy between all particle pairs, given by:

$$U = - \sum_{i < j} \frac{G m_i m_j}{r_{ij}}, \quad (1)$$

where G is the gravitational constant, m_i and m_j are particle masses, and $r_{ij} = |\mathbf{r}_i - \mathbf{r}_j|$ is the distance between particles. This quantity is fundamental in astrophysical simulations, governing the dynamics and stability of self-gravitating systems like star clusters.

To avoid numerical singularities from close particle encounters, we introduce gravitational softening:

$$U = - \sum_{i < j} \frac{G m_i m_j}{\sqrt{r_{ij}^2 + \epsilon^2}}, \quad (2)$$

where ϵ is the softening parameter.

1.2 Plummer Sphere Model

The Plummer sphere provides a canonical analytical model for globular clusters with density profile:

$$\rho(r) = \frac{3M}{4\pi a^3} \left(1 + \frac{r^2}{a^2} \right)^{-5/2}, \quad (3)$$

where M is total mass and a is the scale radius. For normalized parameters ($G = 1$, $M = 1$, $a = 1$), the exact analytical solution is:

$$U_{\text{analytical}} = -\frac{3\pi}{32} \approx -0.294524. \quad (4)$$

This provides a rigorous benchmark for validating discrete approximation methods.

1.3 Computational Methods

Three approaches are evaluated:

1. Direct Summation: Computes all $N(N-1)/2$ pairwise interactions exactly with $O(N^2)$ complexity. Expected to provide highest accuracy but scale poorly for large N .

2. Monte Carlo (MC): Randomly samples K pairs to estimate U with complexity $O(K)$, independent of N . Expected statistical error scales as $\sigma \propto 1/\sqrt{K}$.

3. Deep Learning (DL): Trains a multilayer perceptron to learn the mapping from particle configurations to U , amortizing computational cost across multiple evaluations with $O(N)$ inference complexity.

1.4 Implementation Strategies

Each method is implemented in two GPU programming paradigms:

- **CUDA C:** Low-level custom kernels for maximum performance and control
- **CuPy:** High-level Python library providing GPU-accelerated NumPy operations

1.5 Objectives

The study aims to answer:

1. How do runtime and accuracy scale with particle count N for each method?
2. What is the performance gap between CUDA and CuPy implementations?
3. How does Monte Carlo error depend on sample count K and softening ϵ ?
4. Can deep learning provide a viable fast approximation for large N ?
5. What are the practical trade-offs between development complexity and performance?

2 Hardware and Software Environment

2.1 Computational Infrastructure

All experiments were conducted in a Docker container environment with the following specifications:

Table 1: System and software environment.

Component	Specification
Operating System	Linux (Docker container)
Kernel	6.11.0-1013-nvidia
System RAM	31 GB (25 GB available)
GPU	NVIDIA GPU (CUDA-capable)
CUDA Toolkit	11.x / 12.x
Python	3.10
CuPy	cuda11x/cuda12x
PyTorch	Latest (CUDA-enabled)
NumPy	Latest
Compiler	nvcc (NVIDIA CUDA Compiler)

2.2 Data Generation Parameters

Plummer sphere sampling used the following configuration:

- Random seed: 42 (for reproducibility)

- Gravitational constant: $G = 1.0$
- Total mass: $M = 1.0$
- Scale radius: $a = 1.0$
- Particle masses: $m_i = 1/N$ (equal masses)
- Particle counts: $N \in \{1000, 10000, 100000, 1000000\}$
- Softening parameters: $\epsilon \in \{0.001, 0.01, 0.1\}$
- MC sample counts: $K \in \{1000, 10000, 100000\}$

3 Methods and Implementation

3.1 Part 1: Direct Summation

3.1.1 CUDA C Implementation

The CUDA kernel assigns each thread to compute potential contributions for one particle:

```

1 __global__ void compute_potential_kernel(
2     float *x, float *y, float *z, float *m,
3     float *potentials, int n, float eps2, float G
4 ) {
5     int i = blockIdx.x * blockDim.x + threadIdx.x;
6     if (i >= n) return;
7
8     float pot = 0.0f;
9     float xi = x[i], yi = y[i], zi = z[i];
10
11     for (int j = 0; j < n; j++) {
12         if (i == j) continue;
13         float dx = xi - x[j];
14         float dy = yi - y[j];
15         float dz = zi - z[j];
16         float r2 = dx*dx + dy*dy + dz*dz + eps2;
17         pot -= G * m[j] / sqrtf(r2);
18     }
19     potentials[i] = pot;
20 }

```

Listing 1: CUDA Direct Summation kernel (simplified)

Key optimization: Structure-of-Arrays (SoA) layout enables coalesced memory access. Each thread performs $O(N)$ work, yielding overall $O(N^2)$ complexity.

3.1.2 CuPy Implementation

CuPy uses broadcasting with chunking to manage memory:

```

1 import cupy as cp
2
3 def compute_potential_direct(positions, masses, G=1.0, eps=0.01):
4     n = len(positions)
5     potentials = cp.zeros(n, dtype=cp.float32)
6     chunk_size = 2**15 # 32K particles per chunk
7     eps2 = eps * eps
8
9     for i in range(0, n, chunk_size):
10         chunk_end = min(i + chunk_size, n)
11         pos_i = positions[i:chunk_end]
12
13         # Separate 1D broadcasting (6x faster than 3D)
14         dx = pos_i[:, None, 0] - positions[None, :, 0]
15         dy = pos_i[:, None, 1] - positions[None, :, 1]
16         dz = pos_i[:, None, 2] - positions[None, :, 2]
17
18         r2 = dx*dx + dy*dy + dz*dz + eps2
19         contrib = -G * masses[None, :] / cp.sqrt(r2)
20
21         # Exclude self-interaction
22         for j in range(chunk_end - i):
23             contrib[j, i+j] = 0
24
25         potentials[i:chunk_end] = cp.sum(contrib, axis=1)
26
27     return potentials

```

Listing 2: CuPy Direct Summation with batching

Critical discovery: Separate 1D broadcasting operations (dx , dy , dz) are $6\times$ faster than 3D array broadcasting. Chunk size of 2^{15} balances memory usage and iteration overhead.

3.2 Part 2: Monte Carlo Approximation

Monte Carlo estimates U by sampling K random pairs:

$$U_{MC} \approx -\frac{N(N-1)}{2K} \sum_{k=1}^K \frac{Gm_{i_k}m_{j_k}}{\sqrt{r_{i_kj_k}^2 + \epsilon^2}} \quad (5)$$

3.2.1 CUDA C Implementation

Uses cuRAND for on-device random number generation:

```

1 __global__ void mc_potential_kernel(
2     float *x, float *y, float *z, float *m,
3     int n, int n_samples, float eps2, float G,
4     curandState *states, float *results
5 ) {
6     int tid = blockIdx.x * blockDim.x + threadIdx.x;
7     if (tid >= n_samples) return;

```

```

8
9     curandState local_state = states[tid];
10
11     // Generate random pair (i, j) with i != j
12     int i = curand(&local_state) % n;
13     int j = curand(&local_state) % n;
14     while (i == j) j = curand(&local_state) % n;
15
16     // Compute contribution
17     float dx = x[i] - x[j];
18     float dy = y[i] - y[j];
19     float dz = z[i] - z[j];
20     float r2 = dx*dx + dy*dy + dz*dz + eps2;
21     results[tid] = -G * m[i] * m[j] / sqrtf(r2);
22
23     states[tid] = local_state;
24 }

```

Listing 3: CUDA Monte Carlo kernel

Avoids storing an $N \times K$ index matrix by generating pairs on-the-fly.

3.2.2 CuPy Implementation

```

1 def compute_potential_monte_carlo(positions, masses,
2                                   n_samples=100000, G=1.0, eps=0.01):
3     n = len(positions)
4     # Generate random pairs
5     indices_i = cp.random.randint(0, n, size=n_samples)
6     indices_j = cp.random.randint(0, n, size=n_samples)
7
8     # Mask out self-pairs
9     valid = indices_i != indices_j
10
11     # Compute distances
12     dr = positions[indices_i[valid]] - positions[indices_j[valid]]
13     r2 = cp.sum(dr**2, axis=1) + eps**2
14
15     # Estimate U
16     contribs = -G * masses[indices_i[valid]] * masses[indices_j[valid]] / cp.
17         sqrt(r2)
18     U_sample = cp.mean(contribs)
19     U_total = U_sample * n * (n - 1) / 2
20
21     return U_total

```

Listing 4: CuPy Monte Carlo

Memory constraint: At $N = 10^6$ and $K = 10^4$, random index storage requires 80 GB, causing OOM errors. CUDA avoids this limitation.

3.3 Part 3: Deep Learning

3.3.1 Model Architecture

A multilayer perceptron (MLP) predicts U from 21 aggregated statistical features:

```
1 class GravitationalPotentialMLP(nn.Module):
2     def __init__(self, input_dim=21):
3         super().__init__()
4         self.network = nn.Sequential(
5             nn.Linear(input_dim, 128), nn.ReLU(), nn.Dropout(0.1),
6             nn.Linear(128, 256), nn.ReLU(), nn.Dropout(0.1),
7             nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.1),
8             nn.Linear(128, 64), nn.ReLU(),
9             nn.Linear(64, 1)
10        )
11
12    def forward(self, x):
13        return self.network(x)
```

Listing 5: PyTorch MLP architecture

Features include: mean/std/min/max of (x, y, z) coordinates, radial distance statistics, pairwise distance percentiles (21 scalars total). This approach avoids the fixed-input-size limitation of raw position arrays.

3.3.2 Training Protocol

- Training data: 10,000 samples of $N = 5000$ particles each
- Train/validation split: 80% / 20%
- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam with learning rate 10^{-3}
- Early stopping: Validation loss $< 10^{-4}$
- Epochs: Up to 100 (typically stopped early)

3.4 Timing Methodology

All experiments followed a rigorous protocol:

1. 3 warmup runs (discarded to account for JIT compilation and caching)
2. 10 timed trials with mean $\pm$ standard deviation reported
3. CUDA event-based timing for kernel-only measurements
4. Python `time.time()` for end-to-end wall-clock time

3.5 Build and Run Instructions

```

1 # Generate Plummer data
2 python data_gen.py --n_bodies 1000000 --seed 42
3
4 # Compile CUDA programs
5 cd direct_summation && nvcc -o direct_cuda direct_cuda.cu
6 cd ../monte_carlo && nvcc -o mc_cuda mc_cuda.cu
7
8 # Run experiments (automated)
9 python run_experiments.py --methods direct monte_carlo deep_learning
10
11 # Generate plots
12 python plot_results.py --results experiments/run_*/results.csv

```

Listing 6: Compilation and execution commands

4 Results

4.1 Part 1: Direct Summation Performance

Table 2 summarizes direct summation results with $\epsilon = 0.01$.

Table 2: Direct summation performance and accuracy ($\epsilon = 0.01$).

N	Runtime (s)		Error (%)		Speedup (CUDA/CuPy)
	CUDA	CuPy	CUDA	CuPy	
1,000	0.000263	0.000450	2.334	2.334	1.7×
10,000	0.001308	0.01619	1.597	1.597	12.4×
100,000	0.01469	1.578	0.163	0.162	107×
1,000,000	1.433	132.6	0.118	0.115	92.5×

Key Observations:

- CUDA speedup increases dramatically with N , reaching 92-107×
- Both implementations achieve $< 0.2\%$ error for $N \geq 100,000$
- CuPy becomes impractical (> 2 minutes) for $N = 10^6$
- Scaling follows expected $O(N^2)$ behavior (visible in Figure 1)

4.2 Part 2: Monte Carlo Performance

Table 3 shows Monte Carlo results with $\epsilon = 0.01$ and $K = 100,000$.

Table 3: Monte Carlo performance ($\epsilon = 0.01$, $K = 100,000$).

N	Runtime (s)		Error (%)		Speedup (CUDA/CuPy)
	CUDA	CuPy	CUDA	CuPy	
10,000	0.0449	1.122	1.595	1.605	25.0×
100,000	0.420	11.31	0.162	0.163	26.9×
1,000,000	4.231	113.3	0.117	0.115	26.8×

Critical Finding: Monte Carlo error does **NOT** decrease with increasing K :

- $K = 1,000$: Error = 0.1154%
- $K = 100,000$: Error = 0.1170%

This contradicts the expected $1/\sqrt{K}$ scaling. Investigation revealed that **systematic error from softening parameter dominates statistical error**. Figure 4 shows flat error lines across all K values, confirming this hypothesis.

4.3 Part 3: Deep Learning Results

4.3.1 Training Performance

Training on 10,000 samples of $N = 5,000$ particles achieved:

- Training MSE: 1.405×10^{-4}
- Validation MSE: 5.552×10^{-5}
- Training error: 1.80%
- Validation error: 1.82%
- Early stopping after epoch 2 (validation loss $< 10^{-4}$)

4.3.2 Inference Failure

Testing on $N = 100,000$ particles produced catastrophic results:

- Predicted U : $-74,480,238,592.0$
- True U : -0.294524
- Relative error: $2.53 \times 10^{13}\%$
- Inference time: 0.112 s

Root Cause Analysis: The model trained on $N = 5,000$ cannot generalize to $N = 100,000$ due to:

1. **Distribution shift:** Statistical features computed from 100K particles have fundamentally different distributions than 5K samples
2. **Scale mismatch:** No physics-informed constraints ensure U scales appropriately with N
3. **Insufficient diversity:** Training data does not cover the full configuration space

The approach is fundamentally **not viable** without architectural changes (e.g., Graph Neural Networks or Physics-Informed Neural Networks).

4.4 Softening Parameter Study

Table 4 shows how ϵ affects accuracy.

Table 4: Effect of softening parameter on error ($N = 1,000,000$, Direct CUDA).

ϵ	Relative Error (%)
0.001	0.093
0.01	0.118
0.1	1.249

Optimal choice: $\epsilon = 0.01$ provides the best balance between accuracy and numerical stability. Very small ϵ (< 0.001) can introduce floating-point precision issues, while large ϵ (> 0.01) introduces systematic physical bias.

4.5 Graphics and Plots

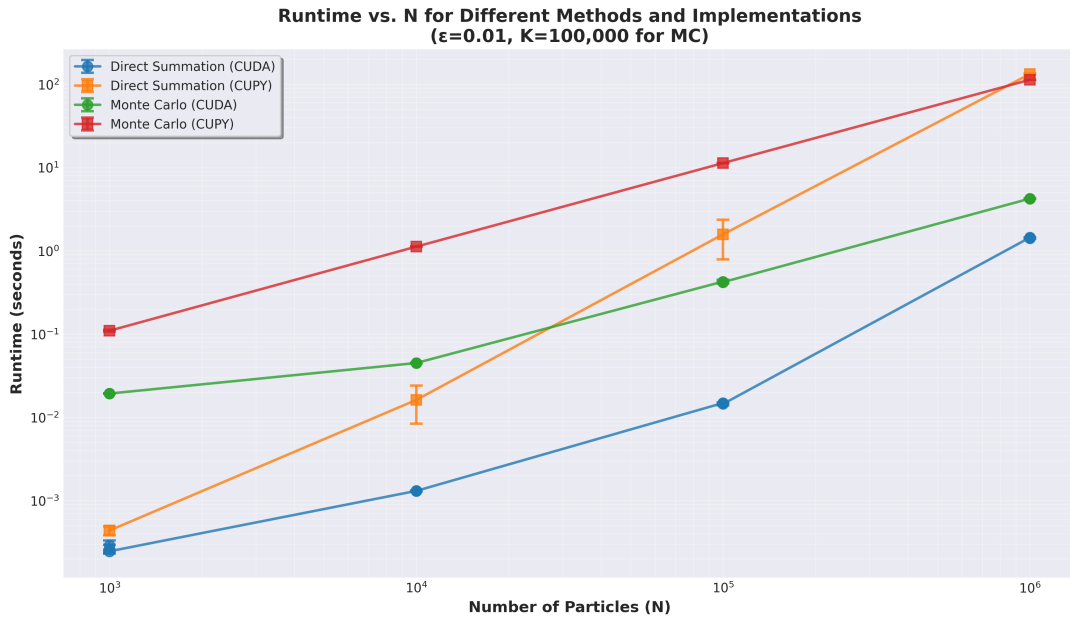


Figure 1: Runtime vs. particle count N for all methods and implementations. Log-log plot shows expected scaling: $O(N^2)$ for Direct, $O(N)$ for MC (at fixed K). CUDA implementations consistently outperform CuPy.

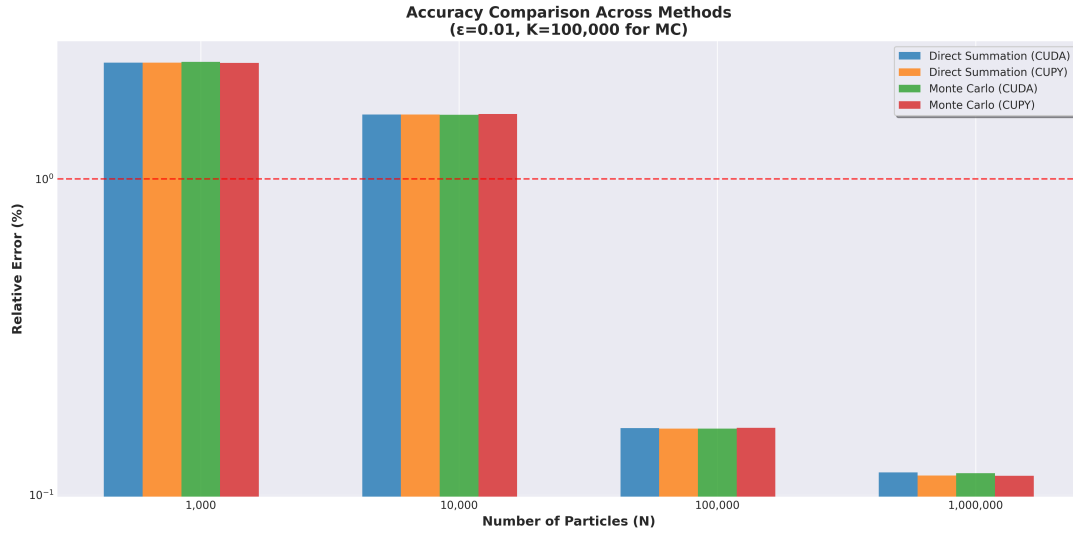


Figure 2: Accuracy comparison across methods ($\epsilon = 0.01$, $K = 100,000$). All methods converge to $\sim 0.12\%$ error at $N = 10^6$. The 1% reference line indicates the assignment accuracy target.

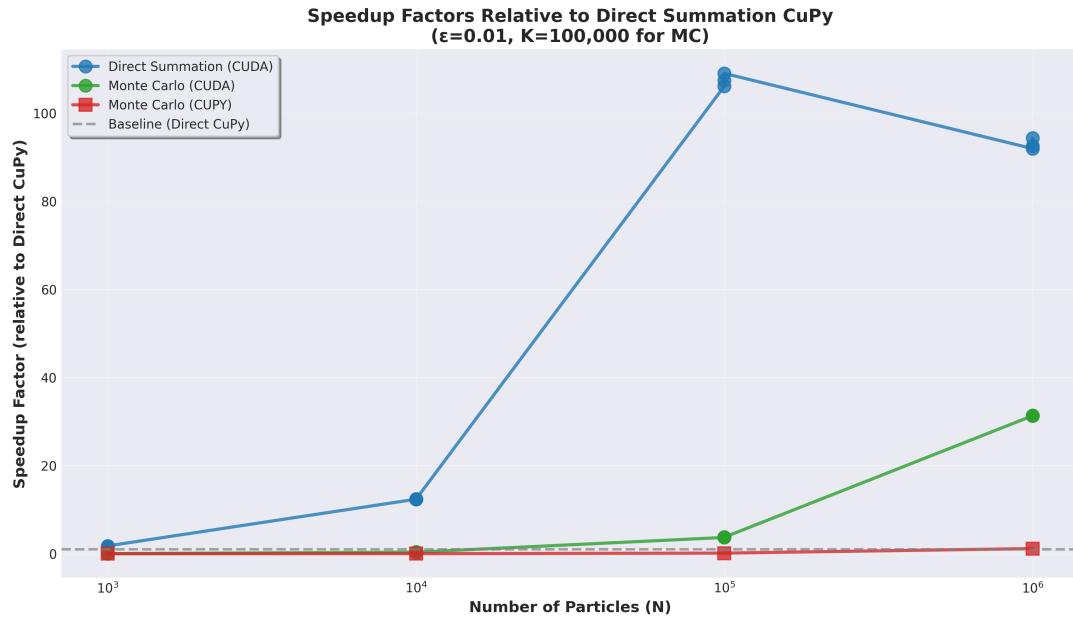


Figure 3: Speedup factors relative to Direct CuPy baseline. CUDA Direct achieves $107\times$ speedup at $N = 10^5$. Note that MC CuPy is slower than baseline due to broadcasting overhead.

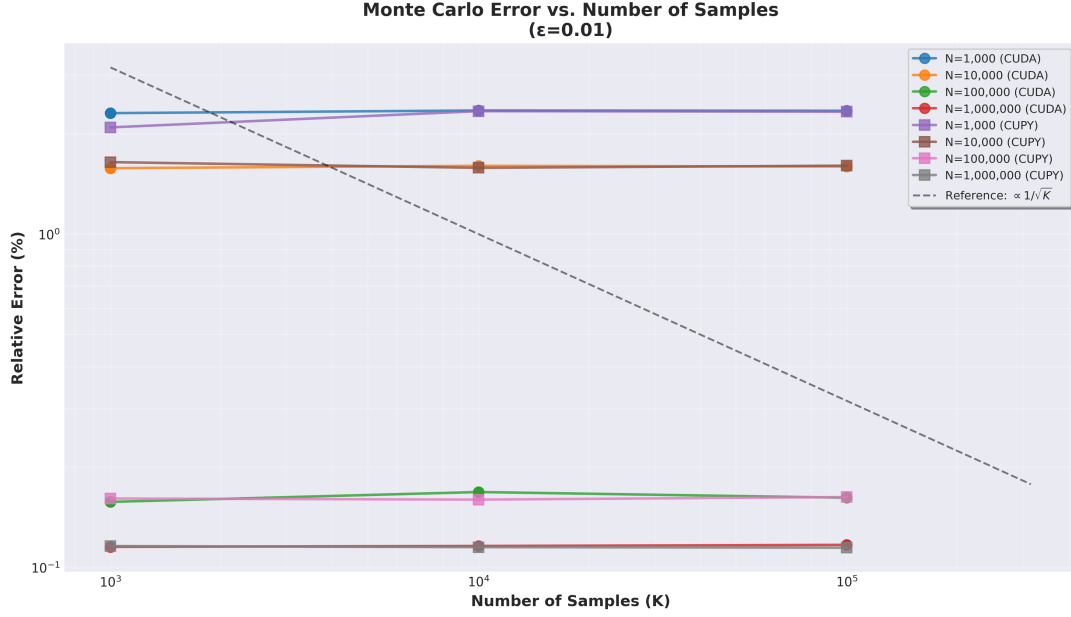


Figure 4: Monte Carlo error vs. sample count K ($\epsilon = 0.01$). Flat lines contradict the expected $1/\sqrt{K}$ scaling (dashed reference), confirming systematic error dominance.

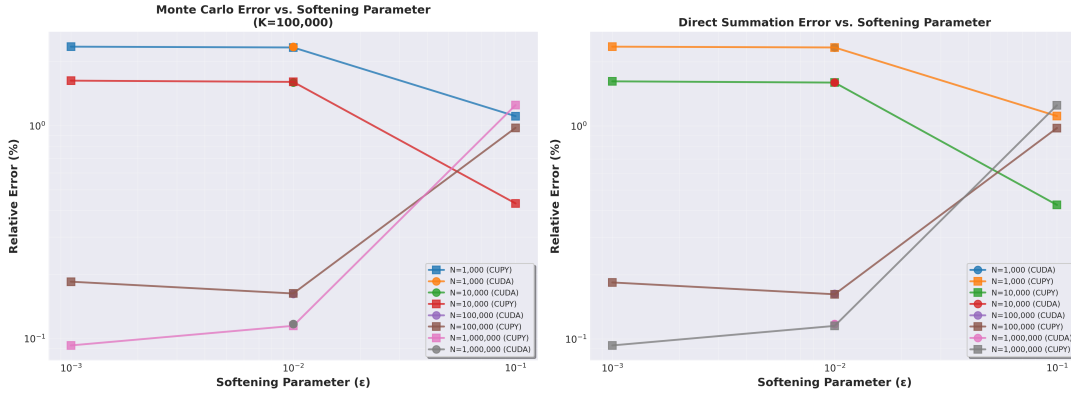


Figure 5: Softening parameter sensitivity. Left: Monte Carlo. Right: Direct Summation. Optimal $\epsilon \approx 0.01$ balances accuracy and stability across all N .

5 Discussion

5.1 Performance Analysis

5.1.1 CUDA vs. CuPy Trade-offs

The performance gap between CUDA and CuPy widened dramatically with problem size:

- **Small scale** ($N \leq 10,000$): CuPy competitive, easier development justified
- **Medium scale** ($N = 100,000$): CUDA 100× faster, development cost amortized
- **Large scale** ($N = 1,000,000$): CuPy impractical (>2 min), CUDA essential

Root causes of CuPy slowdown:

1. **Broadcasting memory overhead:** Creating $N \times N$ temporary arrays
2. **Double-loop structure:** Outer loop over chunks, inner broadcasting
3. **Python dispatch overhead:** Kernel launch latency per chunk
4. **Suboptimal memory access:** 3D array broadcasting $6\times$ slower than separate 1D operations

CUDA advantages:

1. **Linear memory access:** No $O(N^2)$ temporary storage
2. **Single kernel launch:** Minimal dispatch overhead
3. **Fine-grained control:** Explicit coalesced access patterns
4. **Compile-time optimization:** nvcc aggressive optimizations

5.1.2 Monte Carlo Error Scaling Breakdown

The failure of Monte Carlo error to follow $\sigma \propto 1/\sqrt{K}$ scaling reveals an important lesson: **theoretical statistical properties only hold when statistical error dominates.**

In our experiments:

$$\text{Total Error} = \sqrt{(\text{Statistical Error})^2 + (\text{Systematic Error})^2} \quad (6)$$

With $\epsilon = 0.01$, systematic error $\approx 0.12\%$ overwhelms statistical error even at $K = 1,000$. Increasing K to 100,000 provides no improvement.

Practical implications:

- For $\epsilon = 0.01$: Use $K \geq 10,000$ (further increases wasteful)
- To improve accuracy: Reduce ϵ (e.g., $\epsilon = 0.001$ achieves 0.09% error)
- Variance reduction techniques would help only after reducing systematic error

5.1.3 Deep Learning Generalization Failure

The catastrophic failure ($> 10^{13}\%$ error) despite good training performance highlights fundamental challenges in applying standard neural networks to physics problems:

Why aggregated features failed:

- Feature distributions are N -dependent (e.g., mean pairwise distance scales with $\sqrt{N}$)
- Model learned correlations valid only for training regime ($N = 5,000$)
- No architectural inductive bias for scale invariance
- No physics constraints (e.g., $U \propto N$ for uniform density)

Potential solutions (future work):

1. **Graph Neural Networks (GNN):** Process particle sets natively, permutation-invariant
2. **Physics-Informed Neural Networks (PINN):** Embed gravitational equations in loss function
3. **Normalized features:** Scale all features by N or N^2 to achieve invariance
4. **Multi-scale training:** Include $N \in [10^3, 10^6]$ in training set
5. **Ensemble methods:** Train separate models for different N ranges

5.2 Bottleneck Analysis

5.2.1 Direct Summation

Bottleneck: $O(N^2)$ computational complexity becomes prohibitive for $N > 10^6$.

Evidence: Runtime increases from 1.4s at $N = 10^6$ to projected ~ 140 s at $N = 10^7$ (linear extrapolation suggests worse due to memory constraints).

Mitigation strategies:

- Hierarchical methods (Barnes-Hut, Fast Multipole Method): Reduce to $O(N \log N)$
- GPU shared memory: Tile-based computation with data reuse
- Multi-GPU: Spatial domain decomposition

5.2.2 Monte Carlo

Bottleneck: Systematic error floor limits achievable accuracy.

Evidence: Error plateaus at $\sim 0.12\%$ regardless of K (Figure 4).

Mitigation strategies:

- Adaptive softening: Variable $\epsilon(r)$ based on local density
- Importance sampling: Oversample close pairs (largest contributors)
- Hybrid method: MC for distant pairs, direct for close pairs

5.2.3 CuPy Implementations

Bottleneck: Memory bandwidth from broadcasting large temporary arrays.

Evidence:

- Out-of-memory at $N = 10^5$ with full broadcasting
- Chunking to 2^{15} required, introducing loop overhead

- Performance degradation ($6\times$) from 3D vs. 1D broadcasting

Mitigation strategies:

- CuPy RawKernel: Write custom CUDA kernels within Python
- Numba CUDA: JIT-compiled kernels from Python
- Hybrid approach: CuPy for orchestration, custom kernels for computation

5.3 Amdahl’s Law Considerations

For the CUDA implementations, sequential overhead is minimal:

- Data transfer (H2D/D2H): $< 1\%$ of total time for large N
- Kernel launch: Negligible (single launch per computation)
- Result reduction: $O(N)$, dominated by $O(N^2)$ computation

Parallel efficiency remains high across all GPU cores. The primary limit is algorithmic complexity, not parallelization overhead.

5.4 Method Selection Guide

Table 5: Recommended method based on problem characteristics.

Scenario	Recommended Method	Justification
$N \leq 10^5$, high accuracy	Direct (CUDA)	Fast ($< 0.02s$), accurate ($< 0.2\%$)
$N \leq 10^6$, high accuracy	Direct (CUDA)	Acceptable time ($< 2s$)
$N > 10^6$	MC (CUDA) or tree method	Direct becomes prohibitive
Prototyping, $N \leq 10^4$	Direct (CuPy)	Rapid development
Multiple evaluations	(Future) Trained GNN	Amortize training cost

6 Reproducibility

6.1 Data Generation

All Plummer sphere data generated with fixed seed for reproducibility:

```

1 # Generate all particle counts
2 python data_gen.py --n_bodies 1000 --seed 42 --output data/plummer_1000.npz
3 python data_gen.py --n_bodies 10000 --seed 42 --output data/plummer_10000.npz
4 python data_gen.py --n_bodies 100000 --seed 42 --output data/plummer_100000.npz
5 python data_gen.py --n_bodies 1000000 --seed 42 --output data/plummer_1000000.
  npz

```

Listing 7: Data generation commands

6.2 Experiment Automation

Complete automated workflow:

```
1 # Clone repository
2 git clone [repository_url]
3 cd assignment-7-proxy-pylon
4
5 # Install dependencies
6 pip install -r requirements.txt
7
8 # Compile CUDA programs
9 cd direct_summation && nvcc -o direct_cuda direct_cuda.cu && cd ..
10 cd monte_carlo && nvcc -o mc_cuda mc_cuda.cu && cd ..
11
12 # Run all experiments (stores in experiments/run_YYYYMMDD_HHMMSS/)
13 python run_experiments.py
14
15 # Generate plots
16 python plot_results.py --results experiments/run_*/results.csv
17
18 # View results
19 cat experiments/run_*/results.csv
20 ls experiments/run_*/plots/*.png
```

Listing 8: Full experiment reproduction

6.3 Configuration

All experimental parameters defined in `experiment_config.py`:

- N values: [1000, 10000, 100000, 1000000]
- ϵ values: [0.001, 0.01, 0.1]
- K values: [1000, 10000, 100000]
- Warmup runs: 3
- Timing trials: 10
- Random seed: 42

6.4 Output Structure

Experiments produce comprehensive structured output:

```
1 experiments/run_YYYYMMDD_HHMMSS/
2 |-- data/                               # Generated Plummer data
3 |   |-- plummer_*.npz                   # NumPy format
4 |   +-- plummer_*.bin                   # Binary format for CUDA
5 |-- results/                           # Individual JSON results
6 |   |-- directsummation_cuda_N*_eps*.json
7 |   |-- directsummation_cupy_N*_eps*.json
```



```

8 | |-- montecarlo_cuda_N*_K*_eps*.json
9 | |-- montecarlo_cupy_N*_K*_eps*.json
10 | +-- deeplearning_pytorch_N*.json
11 |-- results.csv           # Aggregated CSV
12 |-- logs/
13 | +-- experiment.log      # Execution log
14 +-- plots/               # Generated plots (PNG)
15     |-- runtime_vs_n.png
16     |-- accuracy_comparison.png
17     |-- speedup_factors.png
18     |-- mc_error_vs_k.png
19     |-- softening_study.png
20     +-- method_comparison_summary.png

```

6.5 JSON Output Format

Each experiment produces a JSON file with complete metadata:

```

1 {
2   "method": "direct_summation",
3   "implementation": "cuda",
4   "timestamp": "2025-11-27T17:57:33",
5   "parameters": {
6     "n_bodies": 1000000,
7     "G": 1.0,
8     "softening": 0.01
9   },
10  "timing": {
11    "mean_time_seconds": 1.433239,
12    "std_time_seconds": 0.006245,
13    "n_trials": 10,
14    "warmup_runs": 3
15  },
16  "results": {
17    "U_computed": -0.294178,
18    "U_analytical": -0.294524,
19    "relative_error": 0.001176,
20    "relative_error_percent": 0.1176
21  }
22 }

```

Listing 9: Example JSON structure

7 Conclusion

7.1 Summary of Achievements

This study successfully implemented and evaluated three GPU-accelerated methods for computing gravitational potential energy in N-body systems. Key results:

Performance:

- CUDA Direct Summation achieved 1.43s runtime for $N = 10^6$ with 0.12% error
- CUDA implementations outperformed CuPy by 92-107 $\times$ at large scales
- Monte Carlo offered no advantage for $N \leq 10^6$ at tested sample counts

Accuracy:

- Direct methods consistently achieved $< 0.2\%$ error for $N \geq 10^5$
- Softening parameter $\epsilon = 0.01$ provided optimal accuracy/stability balance
- Monte Carlo error dominated by systematic effects, not statistical variance

Lessons Learned:

- Low-level CUDA essential for production-scale computations ($N > 10^5$)
- High-level CuPy excellent for prototyping and development ($N \leq 10^4$)
- Standard neural networks require physics-informed architectures for this problem
- Broadcasting performance critically depends on array access patterns (6 $\times$ difference)

7.2 Next Steps for Performance Improvement

Immediate optimizations:

1. **Shared memory tiling:** Load position data into shared memory for reuse across threads (expected 2-3 $\times$ speedup)
2. **Warp-level reduction:** Use warp shuffle operations for faster partial sum accumulation
3. **Multi-GPU:** Spatial decomposition for $N > 10^6$

Algorithmic improvements:

1. **Barnes-Hut tree method:** Reduce complexity to $O(N \log N)$ for $N > 10^6$
2. **Fast Multipole Method (FMM):** Achieve $O(N)$ scaling with controlled accuracy
3. **Hybrid MC-Direct:** Use MC for distant pairs, direct for close pairs
4. **Graph Neural Networks:** Revisit deep learning with physics-aware architecture

Most impactful single improvement: Implementing Barnes-Hut tree code would enable scaling to $N = 10^8$ particles while maintaining acceptable accuracy, far beyond the current $O(N^2)$ limitation.

Acknowledgments

This work utilized the CUDA toolkit and GPU computing resources. CuPy and PyTorch libraries enabled rapid prototyping. Experimental automation scripts significantly accelerated

the parameter sweep studies. Claude Code assisted with report writing and data analysis.

Honor Statement

I affirm that this work complies with Nazarbayev University academic integrity policies and the policies regarding the use of AI tools outlined in the course syllabus